

Sakura_Brando の 字符串温故知新

0x00 这次我们讲什么

0x10 哈希：

并不只有字符串 hash，因为 hash 的部分太多太杂，没有人会愿意单独列出来讲，所以这里就由莽某人一起讲了，主要包含数哈希、字符串哈希、数列哈希、树哈希。

0x20 字典树：

包括 0/1 字典树和普通字典树，不准备讲压位字典树，因为这个内容确实没什么好讲的，所以会比较简略，只准备讲一道例题。

0x30 序列自动机

顺便讲一下自动机模型，会简单地一笔带过，因为自动机的大头在省选的 AC 自动机和后缀自动机。

0x40 *KMP：

除了板子，还要讲一些 Border 和 Period 相关的东西，是这次的大头。

0x50 Z 函数

这个就基本上只讲板子，主要应用比较少，而且提高组考纲范围内，但是因为一个并不难，另一方面在提高组的题目中作为正解一种写法的一部分出现过，还是会稍微讲一点。

0x60 manacher

这个除了板子也没什么好讲的，具体应用就是看到不到省选难度的回文串问题直接来，再往上可能要用回文自动机。

我的作业除去板子都是课上的例题，要求刷穿，希望能自己找一些题再刷，该讲稿最后会有一些选做题，因为难度较大，选做题我就不放作业了。

在课件中可能会出现一些和字符串毫不相关的内容，因为没有人会单独拿出来讲，所以莽某人就一起讲掉了。

一些定义可能会出现理论性错误，一般无伤大雅，有错的话课上可以直接提出，大家一起尽量减少课堂上错误存在，课上没有听懂的课后可以来问我，我能答的都会尽量答。

0x01 一些约定

- 默认字符串从 0 开始，若 S 表示一个字符串，则用 S_i 表示 S 的第 i 个字符，当下表是一个范围时，表示将该范围内的字符取出形成的字符串，若 S 为数列同理。
 - 用 $|S|$ 表示 S 的“大小”，如若 S 为字符串则 $|S|$ 表示字符串长度，若 S 为集合则 $|S|$ 为集合元素个数。
 - 默认定义一种加法 $S + a$ 或 $a + S$ ，如 S 是字符串， a 是字符或 S 是数列， a 是一个数，表示一个拼接。
 - 默认 V 表示值域， Σ 表示题目字符集（通常 $|\Sigma| = 26$ ）。
 - 默认 $\oplus$ 表示异或， $\otimes$ 表示一种满足某种要求的运算。
-

0x10 哈希

0x11 数哈希

这个是最常见的哈希之一，常见方法如离散化、随机化哈希等。

离散化

这个东西不一定算做哈希，但实际上这玩意可以看作一种哈希，或者我们可以把它看作广义哈希？因为其本质也是一种压缩映射，将大的实数值域压缩到连续的整数值域，只不过这种哈希的映射方式是无损的，常见用处也就是压缩值域以方便前缀和、树状数组等等被值域限制的算法，实际应用就是看到大值域就想一下能不能离散化，实现过程一般离散化 n 个数是 $\mathcal{O}(n \log n)$ 的，具体实现尽量不要使用大常数的 `map`，使用 `sort + unique + lower_bound` 能简洁优秀地实现，且常数一般较小。

随机化哈希

一般配合随机化算法或者数列哈希使用，常见办法是将待哈希的每个数哈希成随机的大数，以减少因为数据特殊构造产生的随机化算法错误或哈希冲突。

这里可以给两个例题：

给定一个随机生成的整数集合 S ，请在时间复杂度 $\mathcal{O}(|S|)$ ，空间复杂度 $\mathcal{O}(1)$ 的情况下**判断**集合内是否存在一个数出现了奇数次。

给定一些不交的集合 T ，多次询问给定一个集合 S ，**判断** S 是否能够不交地划分成若干个 T 集合的并。

两道题都是随机化哈希中的一个典型——异或判断。

第一个问题中，我们将输入的每个整数随机哈希成一个唯一对应的数，然后全部异或起来，最后看异或和是否为 0，实现简单且时空复杂度均极低。

第二个问题我们沿用第一个问题的思路，构造满足要求的情况异或和为 0，所以可以对于每个集合将每个数随机赋一个权值，使得每个集合的异或和为 0，最后判断给定集合所有数的异或和是否为 0。

我们发现这种随机化哈希配合异或是一种非常使用且便于实现的方法，在遇到判断类问题时可以尝试使用。

0x12 字符串哈希

其实真正光讲字符串哈希没什么意思，最常见的就是进制哈希法，选取一个质数 P ，常见会选择 131 或者 13331，然后将字符串看成是一个 P 进制数，再选取一个模数，或者常见手段是 `unsigned long long` 自然溢出，即对 2^{64} 取模，在不特殊构造的情况下基本能保证不会冲突，是实用的方法。

哈希类问题常用于解决一类经典问题——同构问题。

同构问题，说得简单一点就是问两个东西长得一不一样，常见办法是将待判断的两个结构以相同的方法哈希后判断两者是否相同，或者在通过一些简单的处理后再进行哈希。

常见的比如字符串的循环同构，那么就要介绍字符串的最小表示法。

字符串的最小表示法

给定字符串 S ，求与 S 循环重构的字符串中字典序最小的字符串。

若字符串长度为 n 的 S 和 T 满足 $\exists k \in [0, n), \forall i \in [0, n), S_i = T_{(k+i) \bmod n}$ ，则称 S 与 T 循环同构。

一个非常暴力的想法是将所有与 S 循环同构的字符串取出来比较取最小，这个想法简单自然而优美，但是时间复杂度没法过。

考虑暴力优化，我们可以先预估一个时间复杂度下界，因为字符串输入的时间复杂度瓶颈，我们猜测时间复杂度下界为 $\mathcal{O}(|S|)$ ，而我们一共有 $|S|$ 个待选字符串，可以比较自然地想到均摊每个字符串 $\mathcal{O}(1)$ 解决。

方便处理，我们先将整个字符串复制一份接在后面，即断环成链，若当前我们正在比较 $S_{[a, a+|S|)}$ 和 $S_{[b, b+|S|)}$ ，满足 $S_{a+k} < S_{b+k}$ ，且 $\forall i \in [0, k), S_{a+i} = S_{b+i}$ ，那么我们可以发现任何满足 $j \in [0, k]$ 的 $S_{[b+j, b+j+|S|)}$ 都不可能作为答案，因为对于 $S_{[b+j, b+j+|S|)}$ ，一定可以找到 $S_{[a+j, a+j+|S|)}$ 比它小。

这样的话就非常简单了，维护两个指针 i, j 表示答案待选，暴力比较，若第一次发现 $S_{i+k} < S_{j+k}$ ，则令 $j \leftarrow j + k$ ，可以发现每比较 k 次必然能使 i 指针或 j 指针向右移动 k 步，特别的，若发现 $S_{[i, i+|S|)} = S_{[j, j+|S|)}$ ，则说明字符串 S 有一个长度为 $|j - i|$ 的周期，实际上因为初始我们令 $i = 0, j = 1$ ，此时差为 1，则在差变大的过程中，一定已经将之间的字符串判断过不能成为最小表示法，就可以证明做法正确。

最终可以发现， i 指针与 j 指针都最多向右走 $|S|$ 步，时间复杂度 $\mathcal{O}(|S|)$ 。

我们可以在把字符串转化成最小表示法，再利用字符串哈希压缩成一个数，就可以快速对字符串分类。

例题

这里放一道字符串哈希的例题：

[NOIP2020] 字符串匹配

定义对于字符串 S 的函数 $f(S)$ 表示 S 中出现奇数次的字符的种类数。

定义对于字符串 S ， S^k 表示 S 连续拼接 k 次的结果。

定义对于字符串 A, B ， AB 表示 A 与 B 拼接的结果。

给定字符串 S ，将 S 划分为 $(AB)^k C$ 的形式，要求 $f(A) \leq f(C)$ ，求方案数。

- $|S| \leq 2^{20}$

这道题本次讲课会讲到三次，这里先讲哈希做法。

我们可以先将 AB 看成一个整体 T ，接下来就变成 $T^k C$ 的形式，这个形式比上面美观得多， T 是 S 的一个前缀，而 C 是一个后缀，考虑直接暴力枚举 T 和 k ，时间复杂度是 $\sum_{i=1}^{|S|} \frac{i}{|S|} = \mathcal{O}(|S| \log |S|)$ ，这里不讲调和级数求和证明，具体交给下午的 Qjwj_{233} ，此时我们可以直接得到 C ，接下来就是判定问题，那么首先我们需要确定 $S_{[0, k|T|)} = T^k$ ，即 $S_{[0, k|T|)}$ 有一个长度为 $|T|$ 的周期，这里因为我们暴力枚举 k ，故每次 $k \leftarrow k + 1$ ，我们可以判断 $S_{[(k-1)|T|, k|T|)} = S_{[0, |T|)}$ 。

那么现在问题就来到如何快速判断给定字符串的两个子串相等，这是字符串哈希的拿手好戏，预处理任意前缀的哈希，利用前缀和相减的方式得到区间哈希值判断即可。

剩下的部分就非常 naive，求 T 中有几个前缀 A 满足 $f(A) \leq f(C)$ ，可以预处理每个 $f(C)$ ，再在枚举 T 的过程中利用树状数组统计，总时间复杂度 $\mathcal{O}(|S| \log |S| \log |\Sigma|)$ 。

这样小卡常可以过掉大部分数据点。

再进行一次优化，时间复杂度瓶颈在于暴力枚举 T 和 k ，尝试能不能少暴力枚举一些，发现省略 T 只枚举 k 不太现实，所以可以考虑只枚举 T ，优化 k 的枚举，对此，我们可能需要再挖掘一下题目的性质。

再对题目进行观察，发现 $f(A)$ 是一个奇偶性相关的函数，关注对周期数奇偶分类后的结果，我们可以简单地发现 $f(T^2 C) = f(C)$ ，因为 T^2 中每个字符的出现次数必然是偶数。

那么我们只要求出最大的周期数 k ，然后通过计算 $f(S_{[|T|, |S|]})$ 和 $f(S_{[2|T|, |S|]})$ 就可以得到结果。

我们可以对于每个 T ，二分查找周期数 k 的上界，问题变成如何快速判定对于一个 k 是否满足 $S_{[0, k|T|]} = T^k$ 。

这里需要用到一个 border 相关的结论，先把结论给出。

$$S_{[0, k|T|]} = T^k \iff S_{[0, (k-1)|T|]} = S_{[|T|, k|T|]}$$

具体证明等到后面讲 KMP 时再说，这里先搁置。

那么这样的话我们就将前面枚举的复杂度从 $\mathcal{O}(|S| \log |S|)$ 变为了：

$$\sum_{i=1}^{|S|} \log \left(\frac{i}{|S|} \right) = \mathcal{O}(|S|)$$

这个证明我也留给下午的 $\text{ojw}233$ 了，下午要好好听课。

这样总时间复杂度就可以压到 $\mathcal{O}(|S| \log |\Sigma|)$ ，及时不卡常也能轻松过掉所有数据点。

0x13 数列哈希

这个其实更没什么好讲的，主要分为两种——有序哈希和无序哈希。

有序哈希

对数列进行有序哈希的实质其实是使构造的哈希函数使用一种有序运算，比如说构造哈希函数 $\text{hash}(S)$ 满足：

$$\text{hash}(S + a) \equiv k \text{hash}(S) + a \pmod{P}$$

这个东西是不是眼熟，实际上就是进制哈希法，常用于哈希字符串，常见的 $k = 131$, $P = 2^{64}$ ，实际上我们可以将数列看成广义字符串，将每一个数看成一个字符，实际上就是扩充了字符集（其实就和 C++ 中将 `basic_string` 特化成 `string` 一样）。

无序哈希

这个东西我们也经常会用到，主要用于判断两个集合相等，具体方法是使用一种无序运算，常见的如下：

$$\text{hash}(S + a) = \text{hash}(S) \otimes f(a)$$

其中 $\otimes$ 是一种满足交换律的算法， f 是一个数哈希函数，比如将这个数特化展开以后可以形成平方和、立方异或和等常见形式，但是如果是为了防止被手动构造数据 hack，可以配合上面的**随机化哈希**使用，如使 f 是一个随机化哈希函数，再配合两乃至三种哈希，致力于让出题人看到你的代码都懒的来卡你，恭喜你，你就成功了。

0x13 树哈希

这个东西的用处其实不大，就是用来判断两个树是否同构，我们可以看一下例题：

给定 m 个无根树，第 i 个无根树编号为 i ，对于每个 i ，求与第 i 个无根树同构的无根树中编号最小的无根树的编号。

强调无根树（确信）。

无根的情况下两棵树相同没什么头绪，思考更简单的情况——有根的时候怎么解决。

有根树哈希

对于一棵有根树，两个长得一样的树唯一的差别就是对于一个点儿子的顺序有差别，所以我们现在有两个选择：

1. 将所有儿子的哈希值与自己无序哈希起来作为自己的哈希值。
2. 将所有儿子以一种方式排序完后有序哈希起来作为自己的哈希值。

这两个办法都有自己的好处，我们先讲讲第二种。

第二种方法运用到了有序哈希，我们可以使用更取巧的办法，我们将整棵树压缩成一个字符串，那么有一个现成的东西摆在那里——括号序。

我们可以将整棵树映射成一个括号序，对于每个点，把它的儿子所生成的字符串全部排序完后顺次相接，再在外面包一层括号，直接作为这个子树的哈希输出，这是一种非常好实现且正确率高达 100% 的方法，具体如果要压缩成一个数，可以示显式得到字符串后作字符串哈希，也可以在做的过程中直接进行哈希，哈希成字符串的时间复杂度分析方面，已知对 k 个字符串的比较排序时间复杂度为 $\mathcal{O}(k|S|\log k)$ ，则总时间复杂度为 $\mathcal{O}(n^2 \log n)$ ，如果是直接哈希成一个数，时间复杂度是 $\mathcal{O}(n \log n)$ 。

要注意这里的坑点，尽量使用括号序而要避免前 / 后序遍历，因为我们知道前 / 后序遍历的结果并不能唯一确定一棵树，同样的，用这种方法哈希势必会有产生哈希冲突的可能性。

这里给一种我自己脚捏的哈希法（ $\text{sort}(S)$ 表示将集合 S 排序后的结果， $f(x), g(x)$ 分别是一个关于 x 的随机化函数）：

$$\begin{aligned}\text{hash}'(S + a) &= k \text{hash}'(S) + a \\ \text{hash}(x) &= \text{hash}'(f(\text{size}_x) + \text{sort}(\{\text{hash}(y) \mid y \in \text{son}(x)\}) + g(\text{size}_x))\end{aligned}$$

第一种方法是对于一个点，将它的所有儿子无序哈希起来，这里同样有几个非常容易犯的错误：

使用异或哈希：

因为异或的奇偶性质，常常会产生当一个点有多个本质相同的子树时只能记录其奇偶性，如以下哈希法：

$$\text{hash}(x) = \bigoplus_{y \in \text{son}(x)} k \text{hash}(y) + \text{size}_y$$

它只是简单的将每个子树的哈希值简单处理后异或起来，犯了我们上面的错误。

只使用简单的进制哈希：

当我们的进制哈希与深度相关后，经常会产生令人啼笑皆非的错误：

$$\text{hash}(x) = t + p \sum_{y \in \text{son}(x)} \text{hash}(y)$$

这个哈希实际上等价于：

$$t \sum_{k=0}^n p^k \sum_{i=1}^n [d_i = k]$$

所以你随意构造都能够卡掉。

这里再给一种我脚捏的哈希法 (f, g 定义同上)：

$$\text{hash}(x) = f(\text{size}_x) \left(g(\text{size}_x) + \sum_{y \in \text{son}(x)} \text{hash}^2(y) \right)$$

大家在写哈希的时候，要尽量多用一些奇技淫巧，做到怎么怪异怎么来，如果不放心，大可以上双哈希乃至三哈希，以达到哈希的效果，但是要注意实现常数，别捡了芝麻丢了西瓜，要做到一个不落。

无根树哈希

在我们熟悉有根树哈希后，我们可以手动给无根树定一个根，对于同构的树，我们选取的根应该相同，即根的位置与编号无关，最简单的办法，找树的重心、中心、直径中点都是可以的，我们选取算起来更简单的重心，考虑一棵树最多有两个重心，我们只要将两个重心的哈希值取最小值就好了。

0x20 字典树

0x21 普通字典树

单单一个字典树其实是蛮鸡肋的，我们的 AhoCorasick 曾称字典树为**广义前缀自动机**，实际上这东西板子上也就这么用的，最常见的用处就是判断给定多个文本串，问一个模式串是多少文本串的前缀，这里板子我就不多说了。

具体而言，字典树的用处在于将单串相关问题拓展到多串相关问题，如：

- 暴力匹配 $\Rightarrow$ 字典树
- KMP $\Rightarrow$ AC 自动机
- 后缀自动机 $\Rightarrow$ 广义后缀自动机

这里因为确实普通字典树没什么好讲的，所以就不多说了。

0x22 0/1 字典树

据说 Yandou 昨天已经讲过了一部分，所以这里概念性的东西我也不多说了，这个东西常见就是处理异或问题相关，经常会和异或相关贪心一起出现，常见如给定集合 S ，求 $\max_{y \in S} x \oplus y$ 。

例题

当然，我也是之后才知道，字典树上还可以跑 dp！

[CF1616H] Keep XOR Low

给定可重集 S 和一个正整数 v ，求满足 $T \subseteq S, \forall x, y \in T, x \oplus y \leq v$ 的 T 的个数。

- $|S| \in [1, 150000]$
- $v \in [0, 2^{30})$
- $\forall x \in S, x \in [1, 2^{30})$

首先看到**两两异或**问题，考虑先把集合放在字典树上。

本着求什么设什么的原则，我们设 f_x 表示 x 的子树里满足要求的集合数，然后我们来试着转移一下：

- 当前位为 0 时，我们不能只能在同侧子树中选，则 $f_x = f_{x_0} + f_{x_1}$ (x_0, x_1 分别表示 x 的左右儿子编号)。
- 当前位为 1 时，我们同侧子树中选择没有约束，但是异侧子树之间存在约束，而我们的 dp 状态只能接受一棵子树内部约束，所以得重设状态。

我们重新设一个合理但又不合理的状态，考虑到当前位为 1 时会产生异侧子树之间的互相约束，我们设 f_x 表示 x 子树里满足要求的集合数，再额外设一个 $g_{x,y}$ 表示 x, y 子树里满足要求的集合数（强制要求 $x \neq y$ ），我们可以再考虑转移。

- 若当前位为 0：

$$f_x = f_{x_0} + f_{x_1} - 1$$
$$g_{x,y} = (g_{x_0,y_0} + g_{x_1,y_1} - 1) + (2^{\text{size}_{x_0}} - 1)(2^{\text{size}_{x_1}} - 1) + (2^{\text{size}_{y_0}} - 1)(2^{\text{size}_{y_1}} - 1)$$

- 若当前位为 1：

$$f_x = g_{x_0,x_1}$$
$$g_{x,y} = g_{x_0,y_1} \times g_{x_1,y_0}$$

答案是 $f_{\text{root}} - 1$ 。

我们来一个一个解释转移方程。

首先是 f_x 的转移，当前位为 0 时，我们不能在异侧子树中选数，故要么从左子树选，要么从右子树选，故两边加和，同时因为加和的过程中空集被算了两次。当前位为 1 时，**同侧子树中选数没有要求**，异侧子树互相约束，根据定义即 g_{x_0,x_1} 。

再来看 $g_{x,y}$ 的转移，首先我们应当明确一点， $g_{x,y}$ 的询问中， x, y 子树内部选点之间不存在约束，这样的话两个转移也就很明白了，当前位为 0 时， x 与 y 的异侧不能同时选点，所以是分别将同侧选点的方案加起来，同样的减去 1 的空集贡献，后面部分，因为我们在询问 g_{x_0,y_0} 和 g_{x_1,y_1} 时还会再统计 x_0, x_1, y_0, y_1 **单个子树内部选点的方案数**，所以要保证当前我们统计时不存在只在单侧选点的情况，故统计时还需加上 $(2^{\text{size}_{x_0}} - 1)(2^{\text{size}_{x_1}} - 1) + (2^{\text{size}_{y_0}} - 1)(2^{\text{size}_{y_1}} - 1)$ 。

时间复杂度分析方面，我们发现对于每个节点 x ，它只会被 f_x 遍历或存在一个唯一的 y 被 $g_{x,y}$ 遍历，故时间复杂度正确，为 $\mathcal{O}(n \log |V|)$ 。

有些时候，字典树并不作为正解的一个显式部分出现，但是它可以作为一种建模的思想体现。

[CF1709F] Multiset of Strings (naive version)

定义一个长度为 n 的 0/1 字符串的**可重集合** S 被称为好的当且仅当对于每个长度不超过 n 的 0/1 字符串 s ，集合中以 s 为前缀的字符串的数量不超过 c_s 。

对于每个 c_s ，你确定其为 $c_s \in [0, m]$ 。

给定 n, m 和 f ，求有多少种 c_s 的确定方案使得好集合的最大大小**恰好**为 f 。

- $n \in [1, 15]$
- $m, f \in [1, 1000]$

在看到“0/1 字符串”、“前缀”等关键字后，想到把问题放到 0/1 字典树上描述，那么问题被转化成下面的形式：

你有一个高度为 n 的 0/1 字典树，你可以给每个节点确定一个值 $c_x \in [0, k]$ ，对于每个叶子节点你可以分配权值 v ，对于每个非叶子节点， $v_x = v_{x_0} + v_{x_1}$ ，要求对于所有节点 x 都满足 $v_x \leq c_x$ ，求有多少种 c_x 的确定方案使得 $\max v_{\text{root}} = f$ 。

那么我们就可以考虑 dp 了。

设 $f_{x,k}$ 表示 x 子树内有多少种 c 的确定方案满足 $\max v_x = k$ ，那么我们就可以得到一个比较简单的动态转移方程：

$$f_{x,k} = (m - k + 1) \left(\sum_{i+j=k} f_{x_0,i} \times f_{x_1,j} \right) + \sum_{i+j>k} f_{x_0,i} \times f_{x_1,j}$$

边界条件是 $f_{x,k} = 1, x \in \text{leaf}$ ，特别的，因为我们不能指定 c_{root} ，所以在 $x = \text{root}$ 时要特判。

然后我们会发现，这个状态其实和具体节点无关，即 $f_{x,k}$ 对于同层的 x 的取值是无关的，所以我们可以再对转移方程进行简化，设 $f_{h,k}$ 表示高度为 h 的节点满足 $\max v_x = k$ 的方案：

$$f_{h,k} = \begin{cases} 1 & h = 1, \\ \sum_{i+j=k} f_{h-1,i} \times f_{h-1,j} & h = m, \\ (m - k + 1) \left(\sum_{i+j=k} f_{h-1,i} \times f_{h-1,j} \right) + \sum_{i+j>k} f_{h-1,i} \times f_{h-1,j} & \text{otherwise.} \end{cases}$$

这样就显得更加简单，但是我们还要再优化。

具体实现的过程中，我们可以直接跑 $f_{h,k} = \sum_{i+j=k} f_{h-1,i} \times f_{h-1,j}$ ，然后做类似后缀和的操作，再可以把 h 这一维滚掉，最终时间复杂度 $\mathcal{O}(nm^2)$ 。

因为原题数据范围 $m, f \in [1, 2 \times 10^5]$ ，需要使用卷积加速，这里不作要求。

0x30 序列自动机

字符串里一个小到不能再小的内容，姑且拿出来讲一下，顺便简单地感性讲一下自动机模型。

自动机

以下内容纯属作者自己理解，信口胡诌，如有问题可以现场提出来。

计算机与人在处理问题时的“思维模式”是完全不同的，人在得到一个问题时，会潜意识尝试赋予它一个“意义”，这种意义可以是一种生活场景，或者是一种我们已经熟悉的问题的变形，以便于我们的思考，总而言之，我们希望这个问题尽量**形象化**，因为人总是习惯感性思维，形象化的问题便于感性理解与感性思考。

但是计算机与人不同，现阶段的计算机是死板的，所以计算机比人**更需要模型**，计算机只能接受一组固定的形式并以一种固定的方式得到一个解，计算机需要将问题**抽象化**成一个**数学模型**，便于处理。

自动机就是这样一个对计算机来说很方便的模型，接下来我们来介绍一下自动机模型：

自动机常指**确定有限状态自动机 (DFA)**，由以下五部分组成：

1. 字符集 Σ ：自动机只能输入 $x \in \Sigma$ 。
2. 状态集 Q ：如果将自动机看成一张**有向图**，则 Q 是图中的点集。
3. 转移函数 δ ： δ 接受两个参数 $q \in Q, x \in \Sigma$ ，返回一个值 $\delta(q, x) \in Q$ ，可以写作 $\delta: Q \times \Sigma \rightarrow Q$ ，可以看成有向图中的边。
4. 起始状态 q_0 ： $q_0 \in Q$ ，对应图中的**唯一起点**。
5. 接受状态集 F ： $F \subseteq Q$ ，对应图中的**若干终点**组成的集合。

这个写法可能比较抽象，我们可以一个一个解释这些概念，我们前面讲了字典树，我们用字典树作为一个例子来解释一下，我们看看上面的概念对应到字典树里都是什么。

首先字符集 Σ 就像其字面意思一样，是字典树的所能输入的字符组成的集合，如一般的字典树接受大写字母集合或小写字母集合，0/1 字典树仅接受字符 0, 1，那么这都可以称为字典树这种自动机的字符集 Σ 。

然后是状态集 Q 和转移函数 δ ，乍一眼看到这两个概念可能会想到 dp ，实际上有一定相似性，但不完全一样，就像上面说的，状态集与转移函数对应图中的点集和有向图中的边，在字典树中，这两个概念不难理解，我们每在字典树中插入一个字符串都会产生若干**节点**，每个节点都是一个**状态**，那么字典树中所有节点组成的集合就可以称为状态集 Q ，字典树的每条边都是一个转移函数 δ ，我们思考转移函数的定义 $\delta: Q \times \Sigma \rightarrow Q$ ，再来看字典树的每条边，其实从我们平时的实现都可以看出来，平时我们会写 $ch[p][x]$ 表示节点 p 的走 x 字符能走到的儿子节点，这实际上就是接受一个状态和一个字符得到了一个状态，特别的，若 p 没有 x 这个儿子， $ch[p][x]$ 的值为 0，对于转移函数来说，我们同样可以认为若不存在 $\delta(q, x)$ ，则 $\delta(q, x) = \text{null}$ ，且 null 满足一个性质： $\forall x \in \Sigma, \delta(\text{null}, x) = \text{null}$ 。

最后来看起始状态和接受状态，我们可以先定义**接受**这个概念，前面我们讲到字典树能判断给定模式串是否是文本串集合中任何一个元素的前缀，我们大概能想到自动机的正确使用方法是输入一个**字符串**，那么我们再模拟字典树输入一个字符串后会进行什么：**从根出发**，遍历每个字符并走对应的转移函数若最后停在了一个非空节点，则说明满足要求。

对应自动机概念，我们可以具体地描述一遍整个自动机运行的过程，在这个过程中，我们会将前面的五个概念穿插再其中，对这五个概念有着更深的理解：

首先我们给自动机输入一个字符串 S 满足 $\forall x \in S, x \in \Sigma$ ，即字符串中每个字符都在自动机的**字符集** Σ 内，我们维护一个**当前状态** q_t ，当前状态的初值为**初始状态** q_0 ，接下来我们顺序遍历整个字符串，对于每个字符 x ，我们以当前状态和 x 为参数，使当前状态变更为**转移函数** δ 的返回值，即 $q_t \leftarrow \delta(q_t, x)$ ，遍历完成整个字符串后，若当前状态属于**接受状态** F ，即 $q_t \in F$ ，则称自动机**接受**这个字符串，否则称自动机**不接受**这个字符串。

重新回味这个过程，将字典树重新带入自动机模型中，发现字典树是一种极其特殊的模型，首先字典树的形态是特殊的外向树，其次，字典树的接受状态为全体状态，即 $F = Q$ 。

当然，这里只是初步介绍一点点自动机，可以帮助我们更好地理解之后我们即将学到的另外两个自动机——序列自动机和 KMP 自动机。至于更多的 AC 自动机、后缀自动机等更加高深的自动机就不在我们这堂课的范围内了，所以我也不会对正则语言等更多内容多加赘述。

序列自动机

初步了解了自动机之后我们尝试用自动机的思想去理解序列自动机。

我们定义序列自动机是一个接受且仅接受文本串子序列的自动机，如何去构造这样一个自动机。

对于一个长为 n 的字符串 S ，我们建的序列自动机共 $|S| + 1$ 个状态，至多 $|S||\Sigma|$ 个转移，特别的，为了方便，这里的字符串从 1 开始，则我们将 $n + 1$ 个状态编号为 $0 \sim n$ ，接下来我们写转移函数：

$$\delta(k, x) = \begin{cases} \text{null} & \forall i \in (k, |S|], S_i \neq x, \\ \min\{i \mid i \in (k, |S|], S_i = x\} & \text{otherwise} \end{cases}$$

具体建序列自动机的过程中，我们逆向遍历字符串，维护一个临时数组 t ，其中当遍历到 k 时 $t_x = \min\{i \mid i \in (k, |S|], S_i = x\}$ ，这个过程实际上就是当遍历到 k 时用 k 更新 t_{S_k} 。

接下来定义自动机的初始状态和接受状态，根据我们“接受且仅接受文本串子序列”，以及自动机的构造方法，我们令状态 0 为初始状态 q_0 ，并使全体状态为接受状态。

例题

[HEOI2015] 最短不公共子串 (naive version)

给定字符串 A, B ，求 A 最短的子序列使其不是 B 的子序列。

- $1 \leq |A|, |B| \leq 2000$

发现与 A 和 B 的子序列相关，所以将 A 和 B 的子序列自动机建出来，然后同时在两个自动机上跑 bfs，队列中每个状态是一个三元组 (p, q, len) ，表示当前在 A 的自动机上状态为 p ， B 上为 q ，当前字符串长度为 len ，若存在 $x \in \Sigma$ 使得 $\delta_A(p, x) \neq \text{null}$ 且 $\delta_B(q, x) = \text{null}$ ，则答案为 $\text{len} + 1$ ，否则将状态 $(\delta_A(p, x), \delta_B(q, x), \text{len} + 1)$ 加入队列。

顺便提一嘴正解是再建一个后缀自动机一样的方式 bfs 就好了。

0x40 KMP

上面 0x00 时我们已经说过了，这个是我们这次的大头，出于复习考虑，我们还是简单讲一下这个东西的用处，以及稍微给一下板子。

0x41 KMP 自动机

KMP 用于在线性时间内解决字符串匹配问题，即给定文本串 S 与模式串 T ，求 T 是否以 S 的一个子串的形式出现过，更进一步的，我们可以求出对于 S 的每个前缀，其最长后缀满足是 T 的前缀。

在暴力匹配的过程中，当 S 与 T 匹配时，假设现在我们在匹配 $S_{[k, k+|T|)}$ 和 T ，此时如果失配，我们从 0 开始匹配 $S_{[k+1, k+1+|T|)}$ 和 T ，时间复杂度 $\mathcal{O}(n^2)$ ，我们优化算法的一个非常重要的思想是增加信息的利用率，即不要浪费我们花时间处理出的数据，我们发现假设我们已经匹配出了 $S_{[k, k+i)}$ 与 $T_{[0, i)}$ 相等，接下来我们发现 $S_{k+i} \neq T_i$ ，我们可以尝试通过预处理来使得下次匹配可以直接开始就匹配 S_{k+i} ，那既然 $T_i \neq S_{k+i}$ ，我们可以取 T 更短的前缀 $T_{[0, j]}$ 满足 $T_{[0, j]} = S_{[k+i-j, k+i]}$ ，等量代换成只与 T 相关的式子可得满足条件的 $T_{[0, j]}$ 至少要满足 $T_{[0, j]} = T_{[i-j, i]}$ ，如果能处理这个，我们就可以更快速地做字符串匹配。

单独将这个条件取出来，对于每个前缀 $T_{[0, i]}$ 我们需要得到若干 $j < i$ 满足 $T_{[0, j]} = T_{[i-j, i]}$ ，但是如果真的去维护若干 j 的话可能会很麻烦，我们得在动手计算之前先再多挖掘几个性质。

定义 $F_i = \{j \mid j < i, T_{[0, j]} = T_{[i-j, i]}\}$ ，则 $\forall j \in F_i, F_j \subset F_i$ ，更特别的， $F_i = F_{\max F_i} \cup \{\max F_i\}$ 。

证明：设 $j \in F_i, k \in F_j$ ，则根据定义满足 $T_{[0, j]} = T_{[i-j, i]}, T_{[0, k]} = T_{[j-k, j]}$ ，则通过等量代换可以得到 $T_{[0, k]} = T_{[j-k, j]} = T_{[i-k, i]}$ ，即 $k \in F_i$ ，同时因为 $j \in F_i, j \notin F_j$ ，所以得证第一个结论，第二个结论也非常简单，对于任意 $k < j, k \in F_i, T_{[0, k]} = T_{[i-k, i]} = T_{[j-k, j]}$ ，即 $k \in F_j$ ，那么加上 j 自己就是 F_i ，这些画图出来都可以很好的形象化证明。

有了这个结论我们就可以减少我们的维护量了，对于每一个 i ，我们只需要维护 $\max F_i$ ，即满足要求的最长前缀就好了。方便考虑，我们定义 fail_i 表示 $\max F_i$ ，我们现在就是要在较快的时间内求出整个 fail_i 。

最暴力的方法肯定是对于每个 i 暴力枚举每个可能的 j 暴力匹配，时间复杂度 $\mathcal{O}(n^3)$ 。

时间复杂度瓶颈是暴力枚举和暴力匹配，我们思考如何更快的去计算，就像我们之前说的，优化复杂度的常见方法是增加信息的利用率，我们上面已经找到了优化暴力的一种途径，就是利用我们的 fail 去加速，这里我们一样可以这样做。

该铺垫的都铺垫完了，就该介绍 KMP 算法了。

顺序处理整个字符串，当我们处理到 i 时，我们已经处理完了所有 $k \in [0, i)$ 的 fail_k ，考虑要找到的 fail_i 一定要满足什么要求： $S_{[0, j]} = S_{[i-j, i]}$ ，这个东西与我们上面讲到需要利用 fail 的地方是非常类似的，再一步观察发现， fail_i 的充要条件是 $\text{fail}_i \in \{j + 1 \mid j \in S_{i-1}\}$ ，刚刚我们已经知道遍历 S 的方法了，这样我们就可以得到一种暂时复杂度不明的优化算法：

```

1 for (int i = 2, j = 1; i <= n; ++i) {
2     while (j && s[j + 1] ^ s[i]) j = fail[j];
3     fail[i] = j += s[j + 1] == s[i];
4 }

```

(这应该是 KMP 的最简洁写法了)

我们可以尝试计算一下复杂度，我们发现处理过程中我们主要是在维护两个指针的移动，关注两个指针的移动， i 指针只会向右移动 n 次， j 的右移同理，而 j 的左移最多只会做 n 次，因为每次左移至少左移 1 格，而总共右移量就只有 n 格，证明了时间复杂度是线性的。

得到了 fail 以后，我们就可以做匹配了，匹配的过程与求出 fail 的过程极其相似，实际上是一样的，求出 fail 的过程就是在和自己做匹配，求出 $fail_i$ 的过程就是在询问 $S_{[0,i]}$ 与 $S_{[0,i]}$ 的最长匹配。

实际上具体实现时可以通过将模式串与文本串拼起来以后直接求 fail 来实现简洁地求解。

明白了 KMP 的具体实现与用处，我们考虑如何定义 KMP 自动机。

同序列自动机，我们定义的 KMP 自动机有 $|S| + 1$ 个状态与 $|S||\Sigma|$ 个转移定义如下：

令 $\pi(i) = fail_i$ ，则：

$$\delta(i, x) = \begin{cases} i + 1 & S_{i+1} = x, \\ 0 & i = 0, S_1 \neq x, \\ \delta(\pi(i), x) & \text{otherwise} \end{cases}$$

我们令初始状态为 0，接受状态为 $|S|$ ，则可以得到 KMP 自动机接受且仅接受以 S 为后缀的字符串。

课堂小测：如何修改 KMP 自动机使其接受且仅接受以 S 为子串的字符串？

例题

接下来给两道 KMP 自动机的例题：

[HNOI2008] GT 考试

给定一个字符串 S ，求有多少种长度为 n 的字符串满足 S 不为它的子串。

- $1 \leq |S| \leq 20$
- $1 \leq n \leq 10^9$

设 $f_{i,j}$ 表示做到长度为 i 的字符串，匹配 S 长度为 j 的后缀的方案数，则转移：

$$f_{i,j} = \sum_{x \in \Sigma} \sum_{\delta(k,x)=j} f_{i-1,k}$$

如果写成转移矩阵，则 ($g_{i,j}$ 表示 $f_{k,i}$ 对 $f_{k+1,j}$ 的贡献)：

$$g_{i,j} = \sum_{x \in \Sigma} [\delta(i, x) = j]$$

我们得到了转移矩阵，直接跑矩阵快速幂就好了。

0x42 Period

顾名思义，就是周期，若称字符串 S 有一个 Period k ，则：

$$k < |S|, \forall i \in [k, |S|), S_i = S_{i-k}$$

方便起见，我们称 S 的所有 Period 构成的集合为 $\mathcal{P}(S)$ 。

弱周期引理 Weak Periodicity Lemma

$$p, q \in \mathcal{P}(S), p + q \leq |S| \implies \gcd(p, q) \in \mathcal{P}(S)$$

令 $p < q$ ，设 $d = q - p$ ，我们只要证明 $d \in \mathcal{P}(S)$ ，就可以通过辗转相减法得到结论。

我们分情况讨论：

$$\begin{aligned} \forall i \in [d, |S| - p), S_i &= S_{i+p} = S_{i+p-q} = S_{i-d} \\ \forall i \in [q, |S|), S_i &= S_{i-q} = S_{i-q+p} = S_{i-d} \end{aligned}$$

我们发现， $[d, |S| - p) \cup [q, |S|) = [d, |S|)$ ，命题得证。

注意这里 $[d, |S| - p) \cup [q, |S|) = [d, |S|)$ 的充要条件为 $q \leq |S| - p$ ，移项可得 $p + q \leq |S|$ 。

周期引理 Periodicity Lemma

$$p, q \in \mathcal{P}(S), p + q - \gcd(p, q) \leq |S| \implies \gcd(p, q) \in \mathcal{P}(S)$$

一般情况上面的弱周期引理已经够用了，但上面的弱周期引理是不紧的，现在证明更紧的情况：

因为弱周期定理存在，我们只需要证明 $|S| \in [p + q - \gcd(p, q), p + q)$ 的情况即可，使用数学归纳法来证明该问题。

令 $p < q$ ， $d = q - p$ ，我们先证明 $d \in \mathcal{P}(S_{[0, |S| - p)})$ ， $d \in \mathcal{P}(S_{[p, |S|)})$ ，因为 $d \geq \gcd(p, q)$ ，只要套用上面弱周期引理的证明就好了。

通过数学归纳法， $(q - p) + p - \gcd(q - p, p) \leq |S| - p$ 满足要求，且 $q - p$ 是 $|S| - p$ 的 Period，所以可以归纳证明 $\gcd(p, q) \in \mathcal{P}(S_{[0, |S| - p)})$ ，同理 $\gcd(p, q) \in \mathcal{P}(S_{[p, |S|)})$ ，注意到在归纳的过程中，我们利用了 gcd 的辗转相减法。

已知 $2p = p + q - (q - p) \leq p + q - \gcd(p, q) \leq |S|$ ，移项可得 $p \leq |S| - p$ ，结合上面的证明得到 $\gcd(p, q) \in \mathcal{P}(S_{[0, p)})$ ，同时因为 $\gcd(p, q) \mid p$ ，因为 p 也是周期可以扩展为 $\gcd(p, q) \in \mathcal{P}(S_{[0, 2p)})$ ，再结合 $\gcd(p, q) \in \mathcal{P}(S_{[p, |S|)})$ 得到 $\gcd(p, q) \in \mathcal{P}(S)$ 。

到现在为止就证毕了，但是我们可以试一下能否找到更紧的充要条件，若 $|S| = p + q - \gcd(p, q) - C$ ，在上面的证明的过程中，我们就不能得到 $2p \leq |S|$ ，证明就会失败。

0x43 Border

定义 S 的 Border 集合 $\mathcal{B}(S)$ ：

$$\mathcal{B}(S) = \{S_{[0, i)} \mid i \in [1, |S|), S_{[0, i)} = S_{[|S| - i, |S|)}\}$$

看起来有些眼熟，这和我们前面 KMP 时的 F 定义是类似的，实际上， $F_i = \{|T'| \mid T' \in \mathcal{B}(T_{[0, i)})\}$ 。

那么我们也发现，我们可以将 KMP 时讲到的几个性质都可以套用，我们就得到了 Border 的第一个性质：

$$\begin{aligned} \forall T \in \mathcal{B}(S), \mathcal{B}(T) &\subset \mathcal{B}(S) \\ \mathcal{B}(S) &= \mathcal{B}(\text{longest}(\mathcal{B}(S))) \cup \{\text{longest}(\mathcal{B}(S))\} \end{aligned}$$

那么同样的，我们也可以用 KMP 求出整个 Border 集合，即一直跳 fail 得到的下标集对应的前缀集。

不难发现，在同一个字符串上，即 KMP 的 fail 能够建成一个树的形式，任何一个点到根的路径即是它的 Border 集合，我们称这个特殊的树为**失配树**。

先给一道 Border 的例题：

[NOI2014] 动物园

定义 $\mathcal{B}'(S) = \{T \mid T \in \mathcal{B}(S), 2|T| \leq |S|\}$ ，给定字符串 S ，求：

$$\prod_{i=1}^{|S|} |\mathcal{B}'(S_{[0,i]})| + 1$$

先考虑如果要求的 $\mathcal{B}'$ 改成 $\mathcal{B}$ 怎么做，利用上述的第二性质，我们可以在 KMP 的过程中快速求出 $|\mathcal{B}(S_{[0,i]})|$ 。

稍微挖掘一下 $\mathcal{B}'$ 的性质，定义 $s_i = \text{longest}(\mathcal{B}'(S_{[0,i]}))$ ，首先不难发现 $|\mathcal{B}'(S)| = |\mathcal{B}(\text{longest}(\mathcal{B}(S)))| + 1$ ，那么问题转化成求 f_i ，可以想到其长度至多是 $f_{i-1} + 1$ ，更进一步的， $|s_i| \in \{|T| + 1 \mid T \in \mathcal{B}(s_{i-1})\}$ ，那么我们可以用一种类似维护 fail_i 的方法维护 $|s_i|$ 。

Border 还有和 Period 结合后产生的一些更有用的性质，比如对偶性：

$$S_{[0,k]} \in \mathcal{B}(S) \iff |S| - k \in \mathcal{P}(S)$$

证明可以通过画图解释，也可以从两者的定义解释：

$$\begin{aligned} S_{[0,k]} &\in \mathbb{B}(S) \\ \iff S_{[0,k]} &= S_{[|S|-k, |S|)} \\ \iff \forall i \in [0, k), & S_i = S_{i-|S|+k} \\ \iff |S| - k &\in \mathcal{P}(S) \end{aligned}$$

注意到中间所有变化都互为充要的，故两者之间可以直接转化。

利用这个性质，最重要的一点就是我们可以用 KMP 求字符串的周期。

注意：**Border 的 Border 依旧是 Border，但 Period 的 Period 不一定是 Period!**

例题

那么我们就可以再来看这道题：

[NOIP2020] 字符串匹配

定义对于字符串 S 的函数 $f(S)$ 表示 S 中出现奇数次的字符的种类数。

定义对于字符串 S ， S^k 表示 S 连续拼接 k 次的结果。

定义对于字符串 A, B ， AB 表示 A 与 B 拼接的结果。

给定字符串 S ，将 S 划分为 $(AB)^k C$ 的形式，要求 $f(A) \leq f(C)$ ，求方案数。

▪ $|S| \leq 2^{20}$

上面我们优化哈希的时间复杂度的重要式子：

$$S_{[0,k|T|)} = T^k \iff S_{[0,(k-1)|T|)} = S_{[|T|,k|T|)}$$

这个式子等价于 $|T| \in \mathcal{P}(S_{[0,k|T|)}) \iff S_{[0,(k-1)|T|)} \in \mathcal{B}(S_{[|T|,k|T|)})$ ，我们利用这个性质来二分算出最大循环次数。

那么我们实际上就是要快速询问 S 的一个前缀是否存在一个长为 k 的周期。

我们知道 KMP 能求出最长的 Border，那么就可以推出 KMP 可以求出最短的周期。

接下来又需要用到一个性质：

定义整周期集合 $\mathcal{P}'(S) = \{k \mid k \in \mathcal{P}(S), k \mid |S|\}$ ，则 $\mathcal{P}'(S) \neq \emptyset \iff \min \mathcal{P}(S) \in \mathcal{P}'(S)$ 。

证明采用反证法，我们只需证明前面是后面的充要条件即可，令 $k = \min \mathcal{P}'(S)$ 设存在非整周期 k' 满足 $k' < k$ ，很简单可以发现 $2k \leq |S|$ ，所以我们得到 $k + k' \leq |S|$ ，据弱周期引理可得 $\gcd(k, k') \in \mathcal{P}(S)$ ，而 $\gcd(k, k') < k'$ ， $\gcd(k, k') \mid |S|$ ，故与设的条件不符，故得证。

这样我们就可以得到任意一个前缀的最小整周期，这样我们就可以快速求出每个前缀的最小整周期，从而更新一部分前缀的最大循环次数。

对于另一些前缀，我们发现要么它的最大循环次数为 1，要么就是它自己还有整周期，则所有它循环数次得到的字符串的最小整周期一定是它自己的最小整周期，我们可以从它的最小整周期处计算得到答案，我们发现这样一定能找到最大循环次数，因为容易证得一个字符串的所有整周期都是最小整周期的倍数。

这样的话我们用 KMP 代替字符串哈希也可以将这道题解决掉，时间复杂度仍旧是 $\mathcal{O}(n \log |\Sigma|)$ 。

再来一道例题：

【模板】失配树

给定一个字符串 S ，共 m 次询问，每次询问 S 的两个前缀的最长公共 Border。

- $|S| \in [1, 10^6]$
- $m \in [1, 10^5]$

看题目就能知道这题跟我们上面讲到的支配树有关系，显然，这等价于支配树上的 LCA，可以显式建树，再用倍增、树剖、Tarjan 等方式实现，都是可过的，但是有没有别的做法呢？

引入一个新的定理：

一个字符串的所有 Border/Period 长度可以被划分为至多 $\mathcal{O}(\log |S|)$ 个等差数列。

想要证明这个定理，我们首先证明两个引理：

引理一：一个字符串所有达到字符串长度一半的 Border 构成一个等差数列。

考虑将问题转化到 Period 上，则一个字符串所有不到字符串长度一半的 Period 构成一个等差数列。

通过弱周期引理，我们可以证明所有长度不超过字符串长度一半的 Period 都是最小周期的倍数，同时，最小周期的所有倍数都是字符串的周期，命题得证。

引理二：两个字符串 S, T 满足 $|S| = |T| = n$ ，则 $\{k \mid S_{[0,k)} = T_{[n-k,n)}, k \in [\lceil \frac{n}{2} \rceil, n)\}$ 中的所有元素构成等差数列。

我们发现上一个引理是这个引理的特例，我们可以利用上一个引理来证明，只需要转换到上一个引理的情况。

设上面给出的集合为 K ，则取 $k' = \max K$ ，不难得到 $\forall k \in K, S_{[0,k)} \in \mathcal{B}(S_{[0,k']})$ 。

那么现在问题就转化为证明 $\{t \mid t \in \mathcal{B}(S_{[0,k']}), t \geq \lceil \frac{n}{2} \rceil\}$ 构成等差序列，因为 $k' < n$ ，所以直接转化为引理一的情况，得证。

得到了引理，我们就能通过非常优美的办法构造答案了。

我们递归构造答案，先利用引理一构造后半部分的等差序列，剩余即 $S_{[0, \lceil \frac{n}{2} \rceil)}$ 与 $S_{[n - \lceil \frac{n}{2} \rceil, n)}$ 之间构造，利用引理二构造即可，然后再递归 $S_{[0, \lceil \frac{n}{4} \rceil)}$ 与 $S_{[n - \lceil \frac{n}{4} \rceil, n)}$ 每次规模减半，至多递归 $\mathcal{O}(\log |S|)$ 次即可，得证。

利用这个性质，我们就可以非常简洁地得到另一种做法。

既然 Border 可以被划分成 $\mathcal{O}(\log |S|)$ 个等差数列，且每一段等差数列是连续的，不妨将 Border 按照等差数列的方式剖分，每次往上跳，时间复杂度得到保证，现在的问题转化为如何迅速找到公差和链头。

刚刚我们在证明引理一时顺便证明了公差为最小周期，得益于此，我们对原串做完 KMP 后就可以得到的 fail 数组就是我们的公差，我们令 $p(i) = \min \mathcal{P}(S_{[0,i]})$ 。

类比树剖 LCA 的过程，我们每次将更深的点跳到链头的父亲，但是在这里有些许不同，如果直接将每个等差数列作为链，可能会有链头重合的情况，所以我们认为 k 的链头是 $k \bmod p(k) + k$ ，这样也很好避免了跳过头摸到 0 的情况。

最后就是判断两个点是不是在一条链上，如果两个点 x, y 满足 $x < y, (y - x) \mid p(y)$ ，说明两者在一条链上，则 x 就是答案。

从 KMP 的过程中可以发现， $p(i)$ 随 i 增大是单调不减的，所以不存在更深的点跳过头这种情况，只要比较现在的两个点的大小，将大的往上跳就好了。

实现起来比显示建树更简单，常数也更小。

再给一些杂七杂八的性质：

1. $k \in \mathcal{P}(S), t \geq k, k' \in S_{[0,t]}, k' \mid k \implies k' \in \mathcal{P}(S)$ 。
2. 若 T 在 S 中有两次匹配 k_1, k_2 满足 $k_1 \leq k_2, k_2 - k_1 \leq |T|$ ，则 $k_2 - k_1 \in \mathcal{P}(S_{[k_1, k_2 + |T|]})$ 。
3. 若 $2|T| \geq |S|$ ，则 T 在 S 中的匹配位置构成等差数列。

这几个结论我也不知道有什么用。

0x50 Z 函数

对于字符串 S 定义 $Z(i)$ 表示 $\text{lcp}(S_{[0,|S|-i]}, S_{[i,|S|]})$ ，即 S 的每一个后缀与 S 的最长公共前缀， S 的 Z 函数还可以用于快速求得另一个字符串 T 的每一个后缀与 S 的最长公共前缀（具体实现就像 KMP 一样，为了减少码量可以直接把 T 接在 S 后面）。

我们采用和 KMP 相似的思路，充分利用之前求得的信息，假设当前做到 i ，令 k 表示 $[0, i)$ 中 $i + Z(i)$ 最大的 i ，则 $\forall j \in [0, Z(k)), S_{k+j} = S_j$ ，那么我们就可以初步利用这个信息，得到 $Z(i) \geq \min\{Z(i - k), k + Z(k) - i\}$ ，如果是后面更小，那么 $Z(i)$ 可以尝试进行暴力匹配拓展，若拓展成功，可以得到 $i + Z(i) > k + Z(k)$ ，用 i 更新 k 。

在这个过程中，我们的 k 只有当 $i + Z(i) > k + Z(k)$ 时才会被更新，最多更新 n 次，而不能够更新 k 的 i 对应的 $Z(i)$ 是 $\mathcal{O}(1)$ 计算的，得证时间复杂度 $\mathcal{O}(|S|)$ 。

板子虽然比 KMP 长一点，但是理解起来比 KMP 更加直观。

```
1  z[1] = n;
2  for (int i = 2, l = 0, r = 0; i <= n; ++i) {
3      if (i < r) z[i] = min(z[i - l + 1], r - i);
4      while (i + z[i] <= n && s[i + z[i]] == s[z[i] + 1]) ++z[i];
5      if (i + z[i] > r) r = i + z[l = i];
6  }
```

例题

还是这一道：

[NOIP2020] 字符串匹配

定义对于字符串 S 的函数 $f(S)$ 表示 S 中出现奇数次的字符的种类数。

定义对于字符串 S , S^k 表示 S 连续拼接 k 次的结果。

定义对于字符串 A, B , AB 表示 A 与 B 拼接的结果。

给定字符串 S , 将 S 划分为 $(AB)^kC$ 的形式, 要求 $f(A) \leq f(C)$, 求方案数。

- $|S| \leq 2^{20}$

再考虑找循环的过程, 有没有更简单的做法。

Z 函数来解决这个问题是直观而优美的, 因为对于 T 答案直接就是 $\left\lfloor \frac{Z(|T|)}{|T|} \right\rfloor + 1$, 证明可以根据定义得到, 也可以通过画图得到。

0x60 manacher

这个连例题都不给了, 直接讲板子:

给定一个字符串 S , 求出 S 以所有位置为中心的最长回文串 (也可以是两个字母中间为中心)。

同样围绕着充分利用之前求得信息展开, 受 Z 函数启发, Z 函数中, 最长公共前缀的特点使得我们可以将正在求的点与已经被求得的点关联, manacher 求得是回文串, 观察发现它也可以得到相似的性质, Z 函数中是平移, 而在 manacher 里改成了对称。

首先对字符串做一些操作, 因为现在回文串的中心可以是字符也可以是两个字符中间, 不好维护, 所以我们在每两个字符中间插入一个特殊字符, 再在结尾分别插入一个不同的特殊字符, 减少边界出锅的可能性。

模仿 Z 函数, 我们令 len_i 表示以 i 为中心的回文串中间及右边的最长长度, 维护 k 表示当前 $i + len_i$ 最大的 i , 同样可以得到 $len_i \geq \min\{len_{2c-i}, c + len_c - i\}$, 然后暴力向后匹配, 再尝试更新 k 。

时间复杂度方面和 Z 函数的证明是一样的, 不再说了, 代码也和 Z 函数非常相似。

```
1 for (int i = 1, cen = 1, r = 1; i <= n; ++i) {
2     len[i] = min(len[(cen << 1) - i], r - i);
3     while (len[i] < i && i + len[i] <= n && s[i + len[i]] == s[i - len[i]]) ++len[i];
4     if (i + len[i] > r) r = i + len[i], cen = i;
5     res = max(res, len[i] - 1);
6 }
```

到这里我们的字符串就告一段落了, 后面两个内容讲得比较潦草, 因为实在很少用, KMP 部分主要介绍了 Period 和 Border, 在写题的时候要注意观察, 若发现和这两点有关, 要尽快回想所有相关的性质, 减少时间浪费, 因为大部分结论都比较简单, 就算记不起来考场也不要放弃挖掘性质, 多摸索摸索总会来的, 再不济猜个结论直接上也比空着好。

选做题:

- [\[WC2016\]论战捆竹竿](#)
- [\[CF1286E\] Fedya the Potter Strikes Back](#)
- [\[HNOI2019\] JOJO](#)

最后的最后：

GL & HF